Step selection analysis in three dimensions

Natasha Klappstein, Théo Michelot, Ron Togunov, Joanna Mills Flemming

2026-03-25

In this document, we illustrate the workflow of the analysis of (simulated) three-dimensional tracking data using a step selection function. The steps of the analysis are very similar to the two-dimensional case, but we use functions from the package ssf3D.

```
library(ssf3D) # for data processing and analysis
library(terra) # for raster stack
library(lubridate) # to process dates
library(survival) # to fit the SSF
```

Data

We start from a three-dimensional movement track, which contains x/y/z coordinates at regular time intervals. Here, we do not cover possible pre-processing steps, but note that this is a discrete-time model that assumes regular time intervals between locations. The data frame also has columns for `ID` (track identifier) and `time`.

```
# Load tracking data
tracks <- read.csv("../data/sim_data.csv")
tracks$time <- ymd_hms(tracks$time)
head(tracks)
```

```
  ID        x        y        z                time
1  1 270.0688 248.3955 78.40593 2024-01-01 12:00:00
2  1 268.3511 250.0516 72.66669 2024-01-01 13:00:00
3  1 266.4282 250.2265 68.75363 2024-01-01 14:00:00
4  1 265.3756 247.9307 66.97544 2024-01-01 15:00:00
5  1 264.7936 246.8798 66.16326 2024-01-01 16:00:00
6  1 264.2393 245.4895 65.55775 2024-01-01 17:00:00
```

We also load a three-dimensional environmental variable. This is simulated data, but in a real analysis, this could be something like wind, temperature, or any other spatial variable measured over a three-dimensional grid. In two dimensions, spatial data is typically stored as in raster format. Here, we use a raster stack from the terra package, where the layers range across the third (vertical) dimension. We load the raster stack from a tif file, and set the `depth` attribute to the vertical coordinates of the raster layer (used later to extract).

```
# Load spatial covariate data
r_stack <- rast("../data/3D_raster.tif")
depth(r_stack) <- 1:nlyr(r_stack)
r_stack
```

```
class       : SpatRaster
dimensions  : 600, 600, 300  (nrow, ncol, nlyr)
resolution  : 1, 1  (x, y)
extent      : 0, 600, 0, 600  (xmin, xmax, ymin, ymax)
coord. ref. :
source      : 3D_raster.tif
names       :          1,          2,          3,          4,         5,          6, ...
min values  : 0.02025708, 0.03437005, 0.08967847, 0.01817095, 0.0968914, 0.07737575, ...
max values  : 0.90420640, 0.99916446, 0.93202728, 0.92532647, 0.9260988, 0.94683039, ...
```

Generate random points

We use the function `prep_data()` from ssf3D to generate random points for the step selection analysis. This is analogous to the random (or "control") points used in two dimensions, but here they are three-dimensional points. We specify `distr = "gamma"` to indicate that distances between the start point of each step and the random points follow a gamma distribution (for which parameters are chosen based on the observed data). By default, random points are generated with uniform bearings. The distribution of random points does not represent a strict assumption about the movement of the animals, because movement parameters will be estimated later, but the gamma distribution is simply a way to improve the numerical performance (see Michelot et al., 2024, "*Understanding step selection analysis through numerical integration*", for further details). Importantly, `prep_data()` assumes that the data contains a column called `ID`, which is used to separate independent segments of locations (i.e., between which movement should not be calculated). Therefore, if we have multiple track segements/bursts per individual, we should ensure that `ID` is used to differentiate these.

```r
# Generate random points and get movement variables
data <- prep_data(data = tracks, n_random = 50, distr = "gamma")
data[1:5, 1:6]
```

```
  ID stratum obs        x        y        z
1  1     1-3   1 266.4282 250.2265 68.75363
2  1     1-3   0 262.7006 248.1130 72.32794
3  1     1-3   0 267.4750 249.8414 70.39561
4  1     1-3   0 265.5130 247.1812 72.56053
5  1     1-3   0 266.2162 248.9318 74.74736
```

The new data frame has columns for `stratum`, which groups each observed position with matched random points, and `obs`, which is 1 for observed positions and 0 for random points.

In addition to generating random points, the function `prep_data()` also calculated all movement variables required for the analysis. In particular, `step` is the step length, `omega` is the geodesic length, and `delta` is the geodesic orientation. We will use those three variables later to model step lengths with a gamma distribution and the geodesics with a Kent distribution. Note that this function also has an argument `ssf` which can be set to FALSE; in this case, no random points will be generated and the function will simply calculate the movement variables of observed positions.

Get covariates

We need to extract the spatial covariate at all observed and random positions, and we do this using the function `extract_3d()` from the package ssf3D. This is similar to the function `extract()` in the packages raster and terra, but it also uses the vertical coordinate for a covariate measured in three dimensions.

```r
# Extract spatial covariate values
data$cov <- extract_3d(r_stack, data[,c("x", "y", "z")])
```

We also calculate the movement variable $\sin(\omega)^2 \cos(2\delta)$, which we will include to model geodesics with a Kent distribution.

```r
# Compute transformed variable for Kent distribution
data$omega_delta <- sin(data$omega) * sin(data$omega) * cos(2 * data$delta)
```

Fit step selection function

We fit the step selection function using conditional logistic regression, implemented in the `clogit()` function of the survival package. The formula includes `step` and `log(step)` to model step lengths with a gamma distribution, `cos(omega)` and `omega_delta` to model geodesics with a Kent distribution, and `cov` to estimate habitat selection. We also include the term `strata(stratum)` to specify the grouping of observed and random points.

```
fit <- clogit(obs ~ step + log(step) +
                cos(omega) + omega_delta +
                cov +
                strata(stratum),
              data = data)

fit
```

```
Call:
clogit(obs ~ step + log(step) + cos(omega) + omega_delta + cov +
    strata(stratum), data = data)

                  coef  exp(coef)   se(coef)      z      p
step        -2.018e-02  9.800e-01  2.570e-02 -0.785  0.432
log(step)    1.265e-01  1.135e+00  8.942e-02  1.415  0.157
cos(omega)   9.698e+00  1.628e+04  3.740e-01 25.926 <2e-16
omega_delta  4.927e+00  1.380e+02  2.550e-01 19.320 <2e-16
cov          3.683e+00  3.977e+01  3.918e-01  9.400 <2e-16

Likelihood ratio test=3446  on 5 df, p=< 2.2e-16
n= 49980, number of events= 980
```

The fitted model object includes estimated coefficients and standard errors.

Model outputs

Habitat selection

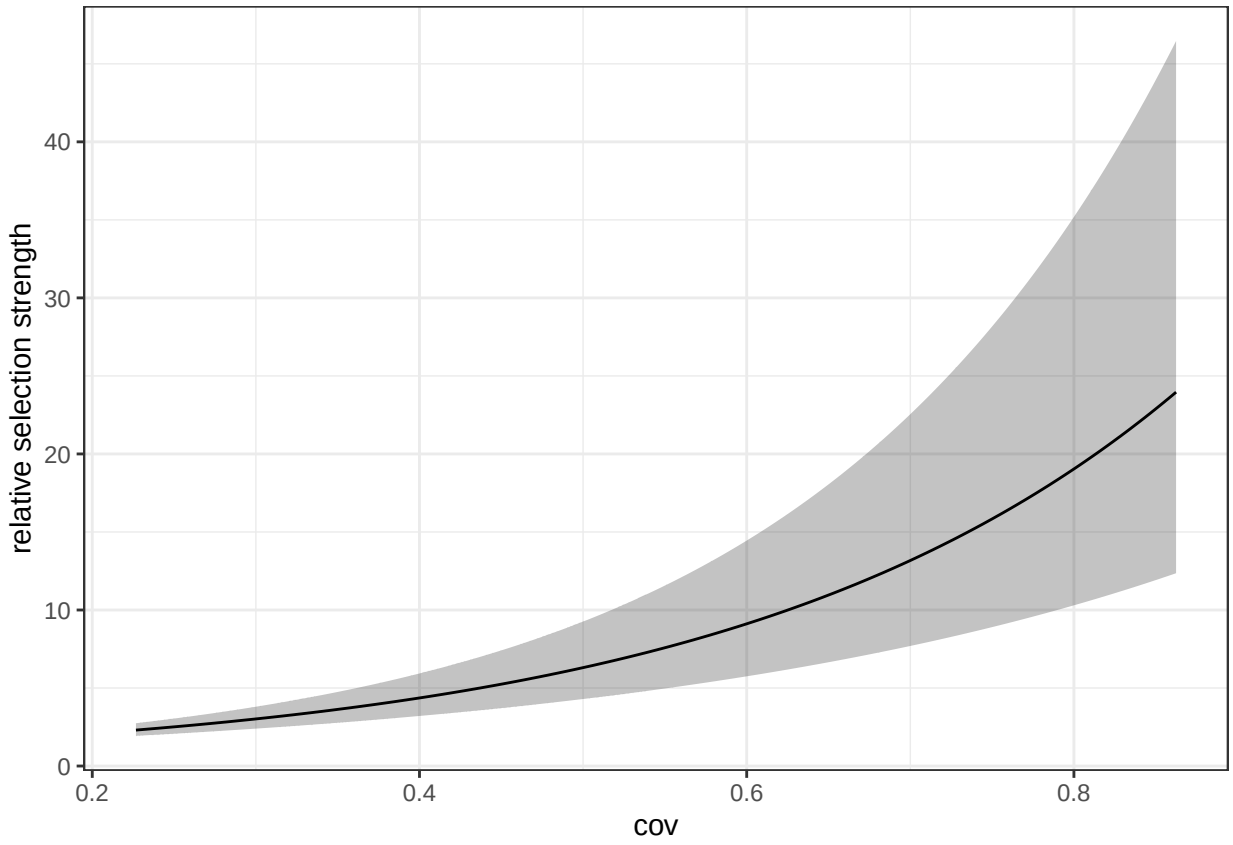
We have one habitat covariate in our model (`cov`). We can interpret its coefficient as relative selection strength; its effect is multiplicative, such that taking the exponential of the estimated β indicates how many times more likely the animal is to take a step if the covariate increases by 1 (keeping everything else constant).

```
exp(fit$coefficients["cov"])
```

```
    cov
39.7668
```

This indicates a positive effect, where the animal is approximately 36 times more likely to take a step with a value of 1 compared to 0 (holding all else equal). The variable `cov` has the range (0, 1) so this corresponds to a change over the entire range. We can also visualise this relationship by using the function `plot_rss()` (note that this will plot relative to 0, by default).

```
plot_rss(model = fit, data = data, variable = "cov", log = FALSE)
```



Movement parameters

Like in two dimensions, the interpretation of movement parameters is not straightforward when random points are not spatially uniform. In this case, the estimated parameters measure a deviation between the distribution of the random points and the distribution of observed movement variables. The ssf3D package includes a function to correct the estimates to account for the distribution of random points, and a function to obtain the estimated parameters

of the implied gamma distribution. There is also a function to plot the estimated gamma distribution over a histogram of the observed step lengths.

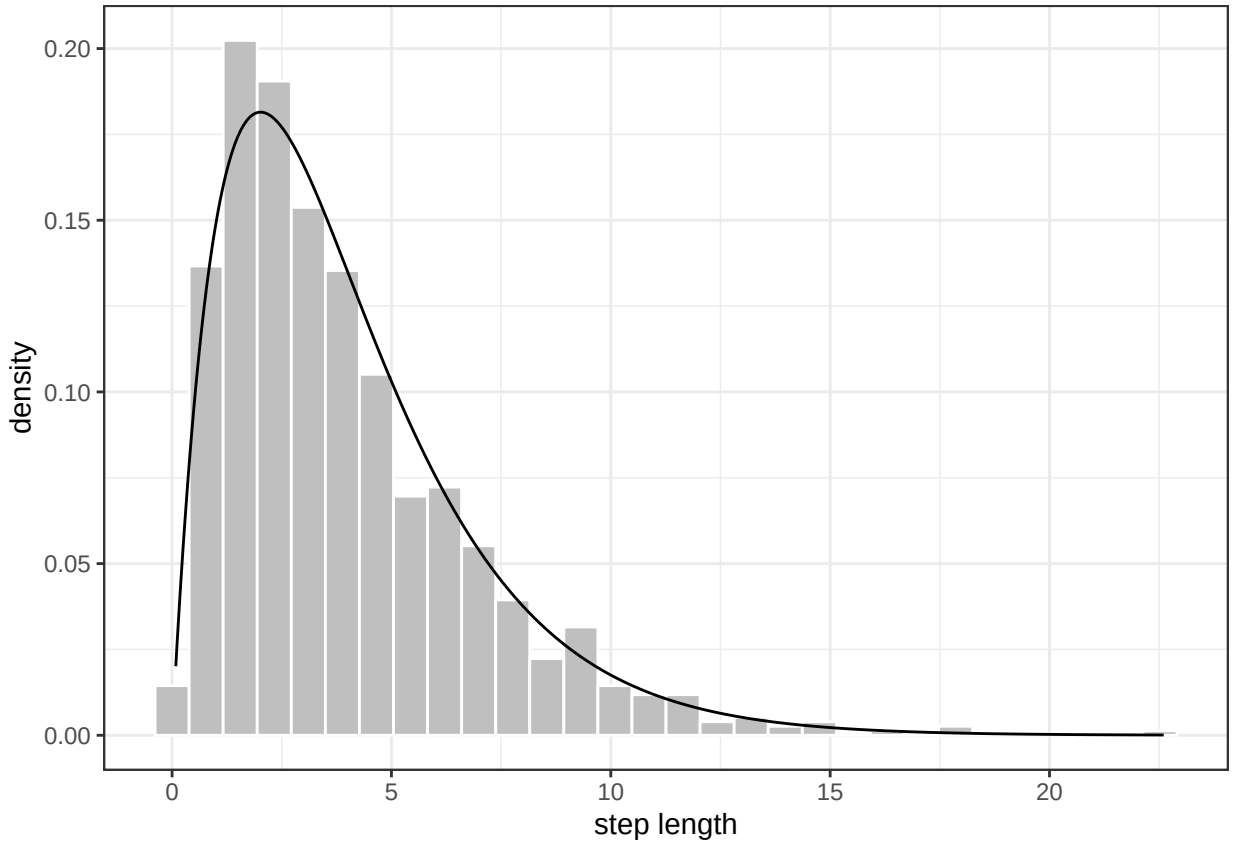
```
# correct coefficients (betas)
beta <- correct_betas(model = fit, data = data)
beta
```

```
       step   log(step)  cos(omega) omega_delta         cov
 -0.4917208  -1.0075503   9.6976299   4.9272734   3.6830323
```

```
# transform (corrected) betas to parametres of the gamma distribution
transform_betas(beta["step"], beta["log(step)"])
```

```
   shape    scale     mean       sd
1.992450 2.033674 4.051994 2.870616
```

```
# plot estimated distribution
plot_gamma(model = fit, data = data)
```



Since we sampled bearings uniformly, there is no correction to be done to the estimated parameters and they directly translate to κ and ρ. Note that `correct_betas()` would have done this correction, if needed.

```r
# estimated kappa
beta["cos(omega)"]
```

```
cos(omega)
   9.69763
```

```r
# estimated rho
beta["omega_delta"]
```

```
omega_delta
   4.927273
```